C# Programs Demonstrating Different Features

a. Function Overloading

Function Overloading is a feature in C# that allows a class to have multiple methods with the same name but different parameters (different number, type, or order of parameters). The compiler automatically calls the correct method based on the arguments passed.

```
using System;

class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }
}

class Program
{
    static void Main()
    {
        Calculator obj = new Calculator();

        Console.WriteLine("Addition of Integers = " + obj.Add(10, 20));
        Console.WriteLine("Addition of Doubles = " + obj.Add(10.5, 20.5));
    }
}
```

b. Single Inheritance

Single inheritance is a type of inheritance where one derived class inherits the features and behavior of only one base class. It is the simplest form of inheritance and promotes code reuse. The derived class can use the members of the base class and also define its own members.

```
using System;

class Animal
{
   public void Eat(){ Console.WriteLine("Animal Eats");}
}
class Dog : Animal
{
   public void Bark(){ Console.WriteLine("Dog Barks");}
}
class Program
{
   static void Main()
   {
      Dog d=new Dog();
      d.Eat();
      d.Bark();
   }
}
```

c. Multilevel Inheritance

Multilevel inheritance is a type of inheritance where a class inherits from another derived class. This creates a chain of inheritance, allowing features to be passed from one class to the next. Each new class can add its own functionality while using the features of its parent class.

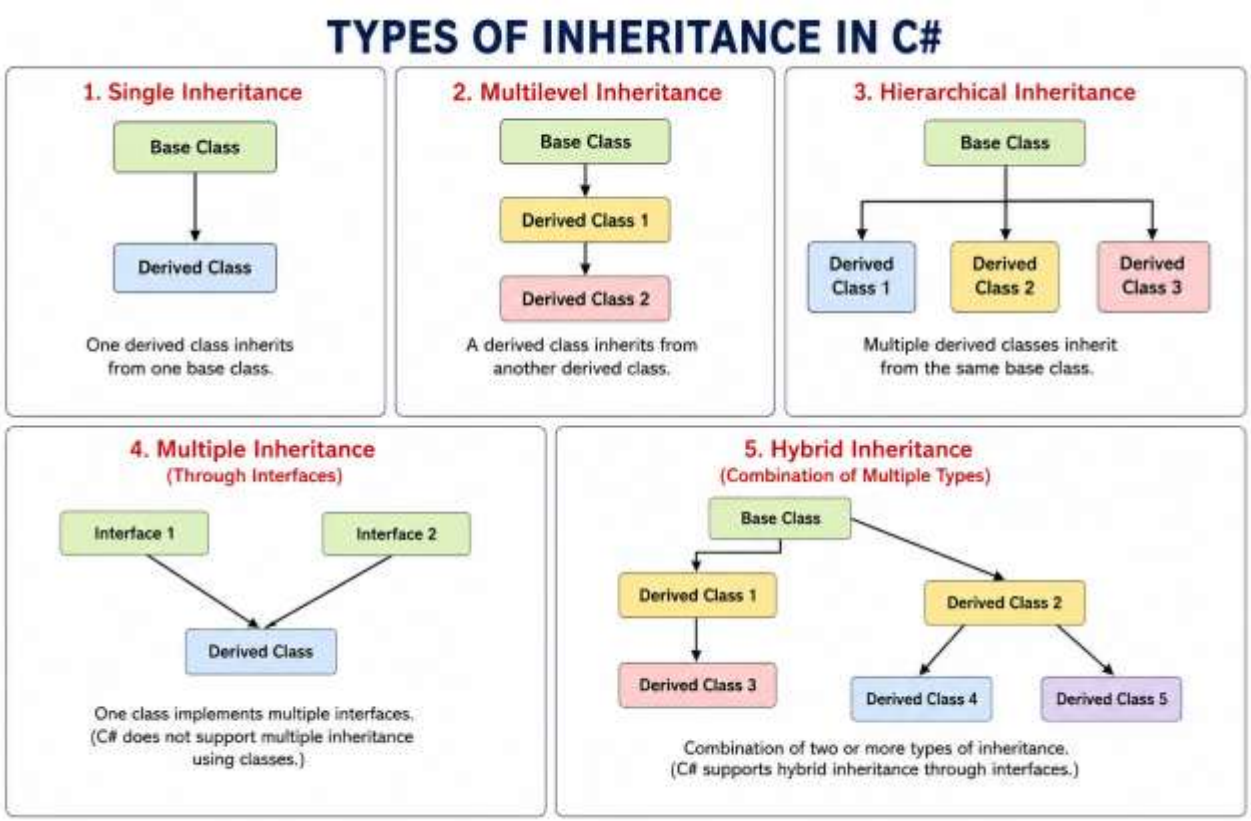
```
using System;
class Animal
{
     public void Eat()
     {
         Console.WriteLine("Animal Eats");
     }
}
class Dog:Animal
{
   public void Bark()
   {
     Console.WriteLine("Dog Barks");
   }
 }
class Puppy:Dog
{
   public void Weep()
    {
       Console.WriteLine("Puppy Weeps");
     }
}
class Program
{
   static void Main()
   {
      Puppy p=new Puppy();
      p.Eat();
      p.Bark();
      p.Weep();
   }
}
```

d Hierarchical Inheritance

Hierarchical inheritance is a type of inheritance where multiple derived classes inherit from the same base class. All derived classes can use the common features of the base class while also having their own unique features. This helps avoid code duplication.

```
using System;

class Animal{ public void Eat(){ Console.WriteLine("Animal Eats");}}
class Dog:Animal{ public void Bark(){ Console.WriteLine("Dog Barks");}}
class Cat:Animal{ public void Meow(){ Console.WriteLine("Cat Meows");}}
class Program
{
   static void Main()
   {
      Dog d=new Dog();
      d.Eat();
      d.Bark();
      Cat c=new Cat();
      c.Eat();
      c.Meow();
   }
}
```



// Note: Multiple inheritance is not supported using classes in C#. Interfaces are used instead.

c. Constructor Overloading

Constructor Overloading is a feature in C# that allows a class to have more than one constructor with the same name (the class name) but different parameter lists. The appropriate constructor is called automatically based on the arguments passed while creating the object.

Properties:

- A class can have multiple constructors.
- Constructors have the same name as the class.
- Constructors must have different parameters (number, type, or order).
- The correct constructor is selected automatically based on the arguments.
- It allows objects to be initialized in different ways (Constructor Overloading).

```
using System;

class Student
{
    public Student()
    {
      Console.WriteLine("Default Constructor");
    }
    public Student(string name)
    {
       Console.WriteLine("Student Name = " + name);
    }
}
class Program
{
   static void Main()
   {
      Student s1=new Student();
      Student s2=new Student("Rahul");
   }
}
```

d. Interfaces

An interface is a blueprint (contract) that defines what a class must do, but does not specify how it should do it. It contains only the declarations of methods, properties, events, or indexers, and the implementing class provides their implementation.

```
using System;
interface IShape
{
   void Area();
}
class Square:IShape
{
   public void Area()
   {
      int side=5;
      Console.WriteLine("Area = " + (side*side));
   }
}
class Program
{
   static void Main()
   {
      Square s=new Square();
      s.Area();
   }
}
```

Properties of interface:

- Defines a contract for classes.
- Cannot be instantiated (no objects can be created).
- Contains method/property declarations only.
- Members are public and abstract by default.
- Implemented using the : operator.
- A class must implement all interface members.
- A class can **implement multiple interfaces**, thereby achieving **multiple inheritance** in C#.
- Cannot have constructors.
- Interface names usually begin with I (e.g., IShape).
- Used for abstraction, loose coupling, and code reusability.

e. Delegates and Events

```
using System;
delegate void Message();
class Test
{
   public event Message ShowMessage;
   public void Display()
   {
      if(ShowMessage!=null)
         ShowMessage();
   }
}
class Program
{
   static void Hello()
   {
      Console.WriteLine("Event Executed");
   }
   static void Main()
   {
      Test t=new Test();
      t.ShowMessage += Hello;
      t.Display();
   }
}
```

This program demonstrates the use of **delegates** and **events** in C#. A **delegate** named Message is declared, which can store the reference of any method that has **no parameters** and **no return value**. The Test class declares an **event** named ShowMessage of type Message. The Display() method checks whether any method is subscribed to the event (ShowMessage != null). If a subscriber exists, the event is raised by calling ShowMessage(). In the Program class, the Hello() method displays the message **"Event Executed"**. Inside the Main() method, an object t of the Test class is created, and the Hello() method is subscribed to the ShowMessage event using the += operator. When t.Display() is executed, the event is raised, causing the Hello() method to execute automatically and display the message on the screen.

Parameter	Method	Delegate	Event
Definition	A method is a block of code that performs a specific task.	A delegate is a type-safe reference that can store the address of one or more methods.	An event is a notification mechanism that uses a delegate to inform subscribed methods when something happens.
Purpose	To execute a particular operation such as calculation, validation, or displaying data.	To pass methods as parameters and invoke them indirectly.	To notify one or more methods automatically when a specific action or event occurs.
Who Calls It?	Called directly by the programmer or another method.	Invoked through the delegate object.	Raised by the class that owns the event and handled by subscribed methods.
Relationship	Performs the actual work.	Points to one or more methods.	Uses a delegate to notify subscribed methods.
Return Value	May return a value or void.	Depends on the method signature it represents.	Does not return a value directly; it triggers subscribed methods.
Keyword Used	No special keyword.	delegate	event
Can Store Multiple Methods?	No.	Yes (Multicast Delegate).	Yes, through the underlying multicast delegate.
Real-Life Example	A teacher explains a lesson to students.	A microphone carries the teacher's voice to speakers.	A school bell rings, and all teachers and students respond automatically.
Software Example	A method calculates the sum of two numbers.	A delegate stores the reference to the Add() method.	A button click event calls methods when the user clicks the button.
Common Uses	Calculations, data processing, business logic, file handling.	Callback methods, asynchronous programming, event handling.	GUI applications, button clicks, keyboard events, mouse events, notifications, alarms.

f. Exception Handling

```
using System;

class Program
{
   static void Main()
   {
      try
      {
         Console.Write("Enter First Number: ");
         int a = Convert.ToInt32(Console.ReadLine());

         Console.Write("Enter Second Number: ");
         int b = Convert.ToInt32(Console.ReadLine());

         int c = a / b;

         Console.WriteLine("Result = " + c);
      }
      catch (DivideByZeroException)
      {
         Console.WriteLine("Cannot divide by zero.");
      }
      catch (FormatException)
      {
         Console.WriteLine("Please enter valid numbers.");
      }
      catch (Exception ex)      // Default catch block
      {
         Console.WriteLine("An unexpected error occurred.");
         Console.WriteLine(ex.Message);
      }
      finally
      {
         Console.WriteLine("Program Finished.");
      }
   }
}
```

Explanation:

1. **try block** – Contains code that may generate an exception.
2. **Specific catch block(s)** – Handle known exceptions such as DivideByZeroException and FormatException.
3. **Default catch (Exception ex) block** – Handles any unexpected exceptions not handled by previous catch blocks.
4. **finally block** – Always executes, whether an exception occurs or not. It is used for cleanup tasks such as closing files, database connections, or displaying final messages.

Why is the Order of catch Blocks Important?

The **specific exception** catch blocks must always be placed **before** the **general (default) exception** block.

- DivideByZeroException and FormatException are **specific exceptions**.
- Exception is the **base (parent) class** for almost all exceptions in C#.

If catch (Exception) is placed first, it will catch **every exception**, and the specific catch blocks below it will **never execute**. This results in a **compile-time error** because the later catch blocks become unreachable.